

Análise Comparativa de Estruturas de Dados: Árvores AVL, Rubro-Negra e Tabelas Hash

Francisco Tiago De Souza Liberato¹, Wellington Santos Rodrigues²

¹Graduação em Ciência da Computação – Universidade Federal do Ceará (UFC)
Campus de Quixadá – 63902-580 – Quixadá – CE – Brasil

tiagoliberato@alu.ufc.br, well.santos@alu.ufc.br

Abstract. *This project consists of the implementation and comparative analysis of a dictionary whose main function is word frequency counting, using four data structures: AVL Tree, Red-Black Tree, Hash Table with Chaining and Hash Table with Open Addressing. All structures and the rest of the program were implemented in C++ and inherit from a common interface. The performance of each structure was evaluated based on the assembly time and the number of key comparisons. Based on these metrics, it was possible to observe that the two hash tables outperformed the other structures.*

Resumo. *O seguinte projeto consiste na implementação e análise comparativa de resultados de um dicionário que tem como principal função a contagem de frequência de palavras, usando quatro principais estruturas de dados: Árvore AVL, Árvore Rubro-Negra, Tabela Hash com Encadeamento Exterior e Tabela Hash com Endereçamento Aberto. Todas as estruturas e o restante do programa foram implementados em C++ e herdam de uma interface em comum. O desempenho de cada estrutura foi avaliado com base no tempo de montagem da estrutura e no número de comparações feitas. Com base nessas métricas foi possível perceber que as duas tabelas hash se sobressaíram em relação às outras estruturas.*

1. Introdução

Os dicionários são estruturas de dados que se caracterizam por serem mutáveis e por guardarem pares chave-valor, onde cada chave é única. Sua complexidade de busca, inserção e remoção se baseia no tipo de estrutura utilizada internamente, podendo ter complexidade de melhor caso $O(1)$ para estruturas como tabelas hash, ou $O(\log n)$ para árvores balanceadas. Essas estruturas são amplamente utilizadas em diversos sistemas computacionais, como sistemas de busca, por permitirem acesso direto a um valor por meio de sua chave. No seguinte projeto, devemos implementar um dicionário que guarde a frequência de palavras em um texto, de modo que a chave seja a palavra e o valor a frequência com que ela aparece no texto.

Para implementar o dicionário foram utilizadas quatro estruturas internas: Árvore AVL, Árvore Rubro-Negra, Tabela Hash com Encadeamento Exterior e Tabela Hash com Endereçamento Aberto. Essas estruturas se diferenciam principalmente na complexidade de suas operações e na forma como guardam os pares chave-valor. As árvores balanceadas possuem complexidade de busca $O(\log n)$, mas ordenam as informações à medida que elas vão sendo inseridas, o que é um diferencial em relação ao problema que está sendo

resolvido. Em contrapartida, as tabelas hash diminuem a complexidade das principais operações para $O(1)$ no melhor caso, mas não mantêm a ordem dos elementos, sendo necessária uma ordenação posterior.

No seguinte projeto, as estruturas herdam de uma interface em comum, o que permite o uso de polimorfismo e consequentemente a escolha da estrutura em tempo de execução pelo usuário. O restante deste artigo está organizado da seguinte forma: a Seção 2 descreve a implementação das estruturas de dados e do programa, a Seção 3 apresenta os testes executados e os resultados obtidos, e a Seção 4 apresenta as conclusões do trabalho.

2. Implementação

2.1. Visão geral das Estruturas

O seguinte dicionário foi implementado na linguagem C++ e utiliza quatro estruturas diferentes para operações com pares chave-valor: Árvore AVL, Árvore Rubro-Negra, Tabela Hash com Encadeamento Exterior, Tabela Hash de Endereçamento Aberto. Todas as estruturas foram implementadas de forma a aceitar valores genéricos com o uso de templates, além de utilizarem as interfaces `Dictionary` e `Iterator` (como mostrado na Figura 1) para fornecer maior flexibilidade ao programa e garantir escalabilidade. A interface `Iterator` é implementada por uma classe interna de cada estrutura, a qual fornece a implementação concreta do iterador.

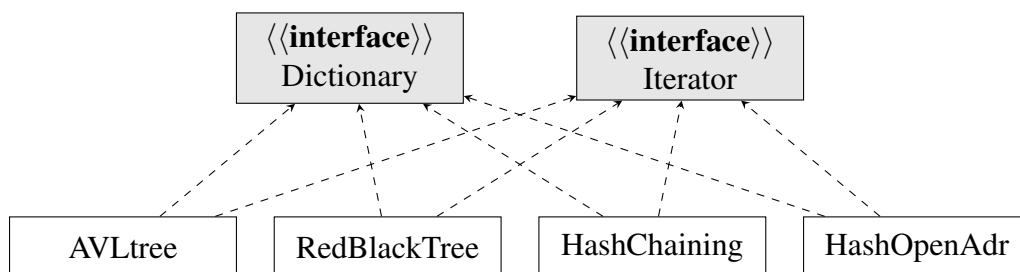


Figura 1. Organização das estruturas de dados

As estruturas de dados foram implementadas como TADs, possuindo métodos públicos que chamam métodos privados que escondem os detalhes da implementação ao usuário. Por utilizarem templates, toda a implementação foi mantida em header.

2.2. Formato de entrada e saída

Os parâmetros usados pelo dicionário são passados diretamente por linha de comandos na hora de executar o programa, sendo eles o nome da estrutura que o usuário deseja usar e o caminho do livro em formato `.txt` que será processado pelo dicionário, como mostrado na Tabela 1. Caso o usuário necessite também poderá acessar um menu de ajuda através do comando `help`, o qual também será acionado se o nome da estrutura informado estiver incorreto.

Tabela 1. Exemplos de execução

Exemplos de uso
<code>./freq dicionario_avl books/livro.txt</code>
<code>./freq dicionario_redblack books/livro.txt</code>
<code>./freq dicionario_chaining books/livro.txt</code>
<code>./freq dicionario_openAdr books/livro.txt</code>

Ao ser informado a estrutura e o caminho do livro corretamente, o programa terá duas saídas diferentes, que incluem:

1. **Resultado:** Arquivo em formato `.csv` com a frequência de cada palavra do livro indicado na execução.
2. **Métricas:** Arquivo em formato `.csv` que guarda as métricas colhidas durante a execução do programa, no caso do presente dicionário, o número de comparações necessárias na construção da estrutura.

2.3. Fluxo de execução

Após a captura da estrutura escolhida e do caminho do livro por meio da linha de comando, a função `main` chama o `Text_Processor` para ler o arquivo do livro e retornar um vetor de strings, logo após adiciona todas essas strings na estrutura escolhida e chama `Text_Processor` novamente para percorrer a estrutura através do iterador e escrever o arquivo de resultados.

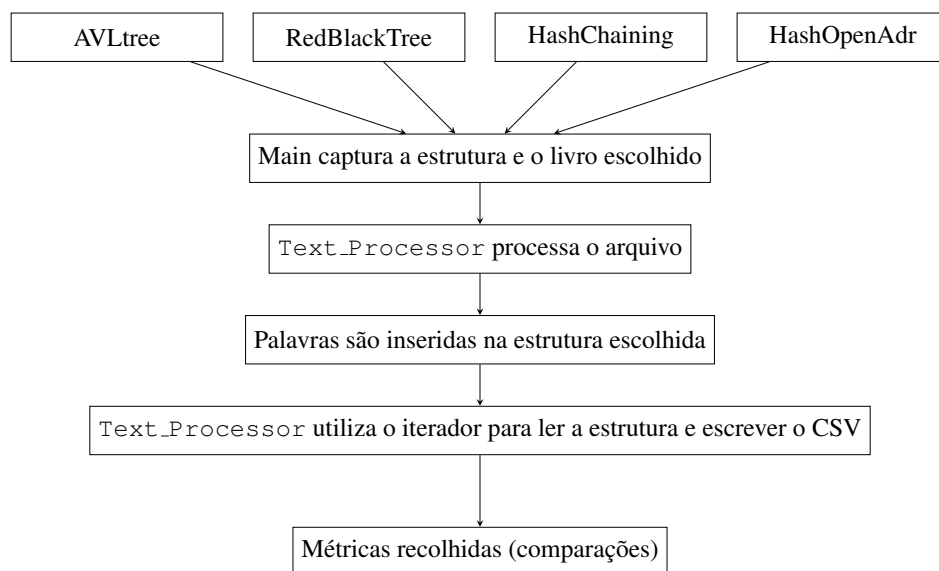


Figura 2. Fluxo de execução do programa

2.4. Processamento de texto

O processamento do texto e escrita dos resultados no dicionário é de responsabilidade da classe `Text_Processor`, que possui apenas métodos estáticos. O método `readFile()` é o responsável por ler o arquivo `.txt` informado na hora da execução

e processá-lo, transformando todas as letras em minúsculas e fazendo limpeza dos caracteres indesejados, utilizando a biblioteca externa `boost.locale`, que auxilia no processamento de caracteres utilizando o padrão UTF-8.

O método `writeCSV()` é responsável por escrever o arquivo com a contagem de palavras, utilizando a interface `Iterator` passada como argumento e especificada em tempo de execução para percorrer a estrutura e colocar os pares chave-valor em um vetor auxiliar. Esse vetor é ordenado alfabeticamente usando o padrão `pt_BR.UTF-8`, para garantir a ordem em caso de estruturas não ordenadas, como a tabela hash, e escrito em um arquivo `.csv` com o formato `palavra, frequência`.

2.5. Árvore AVL

A árvore AVL é uma árvore binária de busca cuja os nós diferem em no máximo o módulo de até uma unidade, sendo denominada de árvore balanceada. Nessa estrutura os nós que seguem essa definição são denominados como regulados, já os nós que superam esse limite são denominados desregulados [Szwarcfiter and Markenzon 2010]. No dicionário os nós da árvore AVL são representados através da seguinte estrutura:

Node
key: K
value: V
height: int
left * right *

Figura 3. Estrutura do nó da Árvore AVL

2.5.1. Inserção

Ao inserir novos nós na árvore devem ser realizadas verificações no seu balanceamento, e em caso de desbalanceamento operações de reestabelecimento são realizadas. Na Árvore AVL, essas operações se baseiam em rotações, as quais têm por objetivo regular cada nó desregulado na árvore. Para tal operação de rebalanceamento existem quatro possíveis rotações, mostradas nas Figuras 4, 5, 6 e 7: rotação direita, rotação esquerda, rotação dupla direita e rotação dupla esquerda. Na árvore AVL utilizada pelo dicionário, a verificação de balanceamento e as rotações são feitas por intermédio do método `fixUpNode()` que realiza rotações até que a árvore esteja totalmente balanceada novamente. A inserção segue os seguintes passos:

1. **Caso Base:** Verifica se árvore está vazia, se estiver o nó inserido será a nova raiz.
2. **Busca:** Percorre a árvore procurando a posição para o novo nó e guarda o caminho em uma pilha, caso o nó já esteja presente lança uma exceção.
3. **Rebalanceamento:** Chama o método privado `fixUpNode()` para verificar se existe algum nó desregulado e aplicar as rotações, caso preciso.

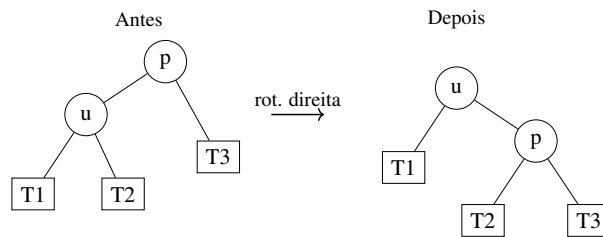


Figura 4. Rotação Simples Direita

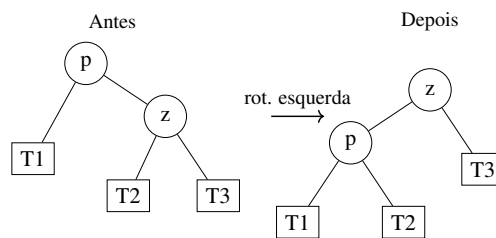


Figura 5. Rotação Simples Esquerda

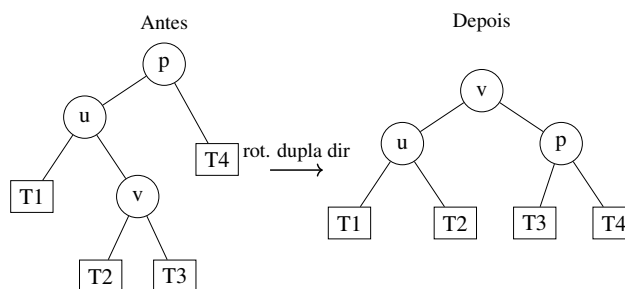


Figura 6. Rotação Dupla Direita

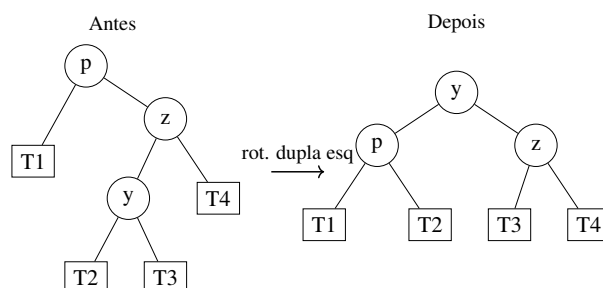


Figura 7. Rotação Dupla Esquerda

Essas rotações seguem alguns casos para serem efetuadas na árvore, os quais são aplicados pelo método `fixUpNode()` e baseiam-se no nó que está sendo inserido:

- **Rotação simples direita (esq-esq):** realizada quando o nó é inserido no filho esquerdo do filho esquerdo do nó desregulado.
- **Rotação simples esquerda (dir-dir):** simétrico do caso anterior, ocorre quando a inserção se dá no filho direito do filho direito do nó desregulado.

- **Rotação dupla direita (esq-dir):** ocorre quando o nó é inserido no filho direito do filho esquerdo do nó desregulado.
- **Rotação dupla esquerda (dir-esq):** simétrico do caso anterior, ocorre quando a inserção se dá no filho esquerdo do filho direito do nó desregulado.

2.5.2. Remoção

A remoção de um nó na árvore AVL, assim como na inserção, implica na verificação do balanceamento e na realização de operações de rotação(ver Figuras 4, 5, 6 e 7) em casos específicos para nós desregulados. No dicionário a verificação e realização dessas operações foram atribuídas diretamente ao método interno `remove()`. A remoção nessa estrutura é iterativa e segue:

1. **Busca o nó:** Busca a chave na árvore e guarda o caminho percorrido em uma pilha, caso não encontre a chave lança uma exceção.
2. **Caso de nó com dois filhos:** Caso o nó encontrado possua dois filhos seu sucessor é buscado, sendo o nó de menor valor da subárvore direita, e tem seu valor copiado para o nó com a chave a ser removida, o novo nó a ser removido será o sucessor encontrado.
3. **Remoção:** O nó final a ser removido tem no máximo um filho, o nó é removido e seu descendente (ou `nullptr`) é colocado no seu lugar.
4. **Rebalanceamento:** Caso algum nó se encontre desregulado, os casos de rotação são aplicados até que toda a árvore esteja balanceada.

2.5.3. Busca

A busca na árvore AVL é feita pelo método privado iterativo `search()`, que percorre a árvore comparando a chave passada como parâmetro com as chaves dos nós, caso encontre retorna um ponteiro para o nó encontrado, se não retorna `nullptr`. Esse método é chamado diretamente por métodos públicos como `contains()`, `get()` e `update()`.

2.5.4. Iterador

O iterador da árvore AVL é implementado através da classe interna `AVLIterator` que herda a interface `Iterator`, fornecendo as implementações concretas de seus métodos. Baseia-se no percurso em ordem, usando uma pilha para guardar todos os nós da subárvore esquerda e capturando o filho direito do nó que está no topo da pilha. No dicionário, o iterador foi implementado seguindo o padrão Java, fornecendo os métodos `hasNext()` e `next()`. Ele pode ser acessado por meio do método público `getIterator()`, que retorna uma instância da classe `AVLIterator`.

2.6. Árvore Rubro-Negra

A árvore Rubro-Negra é uma árvore binária de busca balanceada que possui algumas propriedades que a diferem das demais árvores e tornam-na uma das mais utilizadas nas bibliotecas modernas, como, por exemplo, nas bibliotecas `TreeSet` e `TreeMap` do Java. Essa estrutura precisa satisfazer as seguintes propriedades:

1. Todo nó da árvore deve ser vermelho ou preto.
2. A raiz é da cor preta.
3. O nó NIL é preto.
4. Se um nó é vermelho, então seu filho deve ser preto.
5. Seja um nó v qualquer, todo caminho de v até uma folha possui o mesmo número de nós pretos, denotado por altura negra.

Devido às propriedades, a árvore Rubro-Negra possui campos extras na estrutura de seu nó, além de necessitar de um nó denominado NIL, que substitui o `nullptr` [Szwarcfiter and Markenzon 2010]. No dicionário, para seguir as propriedades, o nó da árvore possui os campos `cor` e `pai`, como observado na Figura 8.

Node	
key: K	
value: V	
color: Color	
parent *	
left *	right *

Figura 8. Estrutura do nó da Árvore Rubro-Negra

2.6.1. Inserção

A inserção na árvore Rubro-Negra segue o mesmo padrão inicial da AVL e também possui complexidade $O(\log n)$, a diferença é que os novos campos dos nós devem ser considerados na inserção de um novo nó. Ao invés de se verificar o balanceamento ao final da inserção, são verificadas as propriedades 2 e 4 referentes à árvore Rubro-Negra. Caso alguma das propriedades for violada, serão realizadas operações de recoloração e rotações iguais às realizadas na árvore AVL (ver Figuras 4, 5, 6 e 7). O método interno `add()` é o responsável por implementar a inserção, seguindo o seguinte algoritmo:

1. **Criação do nó:** Um novo nó vermelho é criado com a chave e o valor passados como parâmetro, tendo seus filhos e pai apontando para o nó NIL.
2. **Busca:** Percorre a árvore comparando chaves e guardando o pai do nó atual, caso a chave já exista uma exceção é lançada.
3. **Inserção:** O novo nó é ligado ao seu pai, sendo inserido à esquerda se sua chave for menor, ou à direita se for maior. Caso a árvore esteja vazia, o novo nó se torna a raiz.
4. **Rebalanceamento:** Chama o método privado `fixUpInsertion()` para verificar se alguma propriedade foi violada e aplicar recolorações e rotações quando necessário.

Na inserção existem casos específicos em que rotações e recolorações são realizadas. No programa, o responsável por aplicar essas operações é o método privado `fixUpInsertion()`, que segue os seguintes casos:

1. O tio do nó pivô é vermelho.

- Se o tio do nó é vermelho, então o avô é preto.
 - Muda a cor do tio e do pai do nó inserido para preto.
 - Muda a cor do avô para vermelho
 - Conserta a propriedade 4 no nó inserido, mas provavelmente o avô virou o nó que quebra a propriedade, passando a ser o nó pivô.
2. O tio do nó pivô é preto e o pivô é o filho da direita.
 - Realiza uma rotação a esquerda no nó pai.
 - O nó pai se torna o novo pivô
 3. O tio do nó pivô é preto e pivô é o filho da esquerda
 - Muda a cor do pai para preto
 - Muda a cor do avô para vermelho
 - Faz uma rotação à direita no avô

2.6.2. Remoção

Como a inserção, a remoção em árvores Rubro-Negras segue uma estrutura inicial básica e tem complexidade $O(\log n)$, mas se torna mais complicada devido aos seus casos de recoloração e rotações. Enquanto na inserção baseamos esses casos no tio do nó pivô, na remoção eles serão baseados no irmão. O método privado `fixRemoveNode()` será o responsável por aplicar as operações a fim de manter principalmente a propriedade 5, uma vez que ao remover um nó preto, a altura negra dos nós será modificada e consequentemente a propriedade 5 será violada. O método interno `RemoveNode()` será o principal responsável por remover o nó, seguindo os seguintes passos:

1. **Busca:** Busca o nó com a chave passada como parâmetro, caso não encontre lança uma exceção.
2. **Caso do nó com dois filhos:** Caso o nó possua dois filhos, seu sucessor é buscado, sendo o menor nó da subárvore direita. Caso contrário, o próprio nó é removido.
3. **Remoção:** O nó final a ser removido tem no máximo um filho, o nó é removido e seu descendente (ou NIL) é colocado no seu lugar.
4. **Rebalanceamento:** Caso o nó removido seja preto, o método `fixRemoveNode()` é chamado para restaurar as propriedades da árvore Rubro-Negra.

Ao remover um nó preto, o método `fixRemoveNode()` é diretamente chamado para correção das propriedades, primeiramente sinalizamos o sucessor do nó a ser removido como duplo preto e ele é o pivô, a fim de manter as propriedades. Seguindo os casos:

1. O irmão é vermelho
 - Troca as cores do irmão com a do pai.
 - Realiza uma rotação a esquerda no pai.
 - Leva a um dos outros casos
2. O nó irmão é preto e seus dois filhos são pretos
 - Pinta o irmão de vermelho.
 - Sobe o duplo preto para o pai.

- Se o pai era vermelho antes das alterações, então as operações acabaram.
 - Se não, trata o nó pai como novo pivô.
3. O nó irmão é preto, seu filho esquerdo é vermelho e seu filho direito é preto
 - Troca as cores do nó irmão com a do seu filho esquerdo.
 - Realiza uma rotação a direita no irmão.
 - Nesse passo, o nó irmão é preto e seu filho direito vermelho, recaímos no caso 4.
 4. O nó irmão é preto, seu filho esquerdo preto e seu filho direito vermelho.
 - Pinta o filho direito do irmão de vermelho.
 - Pinta a cor do irmão com a mesma cor do pai.
 - Pinta o pai de preto.
 - Rotaciona a árvore à esquerda tendo como pivô o pai.
 - Após esse caso nenhuma regulagem será mais necessária.

2.6.3. Busca

A busca na árvore Rubro-Negra se assemelha a busca na AVL, é feita pelo método privado `search()` que compara a chave informada com os nós da árvore e retorna um ponteiro para o nó se encontrar, se não retorna NIL. Essa busca serve como base para outros métodos públicos e privados, como `RemoveNode()`, `update()`, `contains()` e `get()`.

2.6.4. Iterador

O iterador da árvore Rubro-Negra segue a mesma lógica do `AVLIterator`, sendo implementado pela classe `RBIterator` que herda da interface `Iterator` e implementa de forma concreta os métodos `hasNext()` e `next()`. Esse iterador usa uma pilha para guardar os nós da sub-árvore esquerda e depois desempilha um por um e empilha o filho direito de cada um, caso possua. Pode ser acessado pelo método público `getIterator()` que devolve uma instância de `RBIterator()`.

2.7. Tabela Hash com Encadeamento Exterior

A tabela hash é uma estrutura de dados que tem como principal objetivo diminuir a complexidade de Inserção, remoção e busca, sendo uma estrutura que no melhor caso possui complexidade $O(1)$ e no pior caso $O(n)$ para as suas principais operações. Essa estrutura resolve o principal problema relacionado ao armazenamento de informações em tabelas na memória, ao invés de usar o valor direto no seu índice de memória, mapeamos esse valor com base em sua chave para um bloco de memória limitado que vai de 0 a $m - 1$. Uma chave é diretamente atrelada a um valor, consideramos essa chave como um valor numérico, essa chave irá ser processada por uma função de hashing que retornará um índice no intervalo do bloco de memória e que poderá ser utilizado para inserção, busca e remoção de valores [Cormen et al. 2022].

Ao lidar com tabelas de dispersão, a função de hashing pode atrelar chaves diferentes ao mesmo índice, o que acontece devido ao uso da operação *mod* para garantir que

o índice esteja no intervalo de 0 a $m - 1$, nesses casos existem duas maneiras possíveis de lidar com esse problema, a primeira delas é a **técnica de encadeamento exterior**, que transforma a tabela em um vetor de listas. Se ocorrer uma colisão de índices, o par chave-valor será guardado diretamente no final da lista encadeada daquele índice, como mostrado na Figura 9. Ademais, para garantir uma tabela de tamanho eficiente, um fator de carga é delimitado, o qual é calculado por *Quantidade de elementos*/ m , esse fator é estabelecido diretamente no construtor da estrutura.

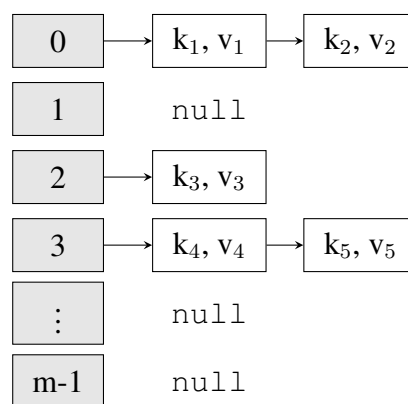


Figura 9. Estrutura da Tabela Hash com Encadeamento Exterior

2.7.1. Hashing

No dicionário, o hashing é realizado por meio da struct `hasher` que possui duas especializações do algoritmo `FNV-1a` que sobrecarregam o operador `()`, a primeira é uma implementação genérica que percorre os bytes e realiza uma operação de XOR e depois multiplica-os por uma constante, a segunda é especializada para strings, aplicando as mesmas operações sobre os bytes de seus caracteres, veja o funcionamento nas Figuras 11 e 10.

Para aumentar a flexibilidade em caso de chaves com estruturas próprias, a struct de `hasher` foi aplicada no template, permitindo que o usuário crie sua própria função de hashing e a passe como parâmetro para a estrutura. O método privado `hash_code()` será o responsável por chamar `hasher` e aplicar a operação de `mod` no índice retornado para garantir que ele está no intervalo da tabela.

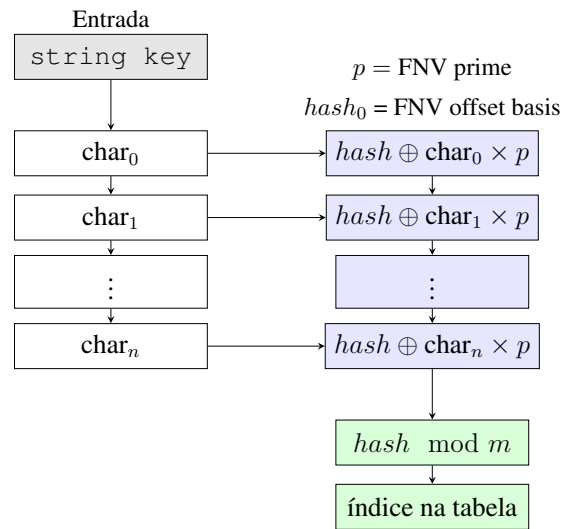


Figura 10. FNV-1a aplicado à chave do tipo `string`

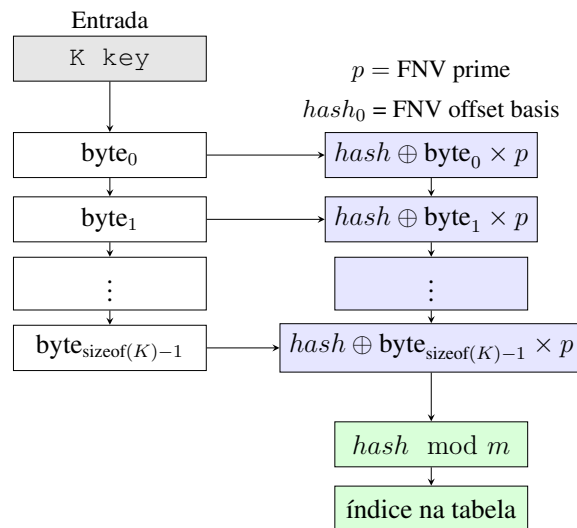


Figura 11. FNV-1a aplicado à chave do tipo genérico `K`

2.7.2. Inserção

A inserção na tabela hash com encadeamento exterior tem complexidade de melhor caso igual a $O(1)$ e $O(n)$ no pior caso em que n inserções colidem no mesmo índice. O método interno de inserção `add()` é responsável por verificar o fator de carga e aumentar a tabela caso necessário, seguindo o algoritmo:

1. **Busca:** Verifica se a chave já está presente na tabela usando o método `contains()`, se sim, lança uma exceção.
2. **Fator de carga:** Verifica se o fator de carga ultrapassou o limite estabelecido, se sim, chama o método `rehash()` para criar uma tabela maior e distribuir os elementos da antiga tabela para a nova.
3. **Inserir:** Calcula o hash code através do método `hash_code()` e insere ao final da lista que está naquele índice.

2.7.3. Remoção

A remoção é um pouco mais simples que o algoritmo anterior, visto que não se preocupa com o fator de carga da estrutura, possuindo complexidade $O(1)$ no melhor caso e $O(n)$ no pior caso quando todos os números foram mapeados no mesmo índice. O algoritmo de remoção é aplicado pelo método interno `remove_pair()`, seguindo os seguintes passos:

1. **Busca:** Verifica se a chave está presente na tabela usando o método `contains()`, se não, lança uma exceção.
2. **Remover:** Calcula o hash code através do método `hash_code()` e remove o elemento da lista naquele índice.

2.7.4. Busca

A busca é realizada pelo método privado `search()`, que calcula o índice usando o método `hash_code()` e percorre a lista correspondente comparando as chaves. Caso encontre, retorna o par chave-valor, caso contrário lança uma exceção. A busca possui complexidade $O(1)$ no melhor caso e $O(n)$ no pior caso, sendo limitada pelo tamanho da lista do índice acessado. Esse método serve de base para os métodos públicos `get()` e `contains()`. A busca também é aplicada pelo método `find()` que retorna uma referência para o par chave-valor e é usado pelo método público `update()` para modificação.

2.7.5. Iterador

O iterador da tabela com encadeamento exterior é implementado pela classe privada `HCIterator` que herda da interface `Iterator`. Implementa de forma concreta a interface, percorrendo todos os índices e lendo cada lista, guardando tudo em uma fila para uso dos métodos `hasNext()` e `next()`.

2.8. Tabela Hash com Endereçamento Aberto

A tabela hash com endereçamento aberto, assim como a tabela com encadeamento, é uma estrutura que busca diminuir a complexidade de operações com pares chave-valor. No método de endereçamento aberto, ao invés de manter um vetor de listas encadeadas, mantemos um vetor simples de slots, no qual serão endereçados no máximo m valores. A técnica consiste em remapear a chave novamente em caso de colisão, guardando o número de tentativas usadas para achar um slot de memória. Esse número de tentativas é usado posteriormente em operações de busca, remoção e verificação [Cormen et al. 2022].

Na implementação da técnica de endereçamento aberto para tratamento de colisões, utilizamos uma estrutura específica para cada slot no vetor, que guarda três valores: a chave, o valor e o status, ver essa estrutura na Figura 12. O status possui três valores possíveis: `EMPTY`, `ACTIVE` e `DELETED`, os quais são usados para o remapeamento de chaves em caso de colisão. O remapeamento é realizado pelo método `hash_code()`, que implementa a técnica de **double hashing**, calculando um novo índice a cada tentativa por meio da fórmula:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \mod m$$

onde h_1 é o hash primário, h_2 é o hash secundário e i é o número de tentativas realizadas.

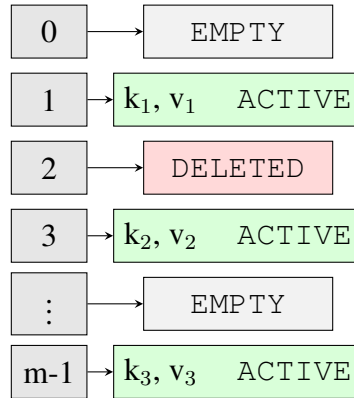


Figura 12. Estrutura da Tabela Hash com Endereçamento Aberto

2.8.1. Hashing

A codificação das chaves nessa estrutura segue exatamente a mesma lógica da tabela com encadeamento, utilizando duas especializações do algoritmo **FNV-1a**, uma para **estruturas genéricas estáticas** e outra para **strings**. O tratamento de colisões é feito pelo método responsável por cada operação, visto que para cada tentativa a struct `Hasher` é chamada novamente dentro do método `hash_code()`, que calcula um novo índice com base no número de tentativas i , usando a técnica de double hashing.

2.8.2. Inserção

Na inserção na tabela hash com endereçamento aberto, o algoritmo segue os mesmos passos da inserção na estrutura anterior, verificando a existência da chave na tabela e o fator de carga. No entanto, nessa estrutura temos a presença do número de tentativas i , que é usado pelo método `insert_internal()` ao chamar o `hash_code()` para calcular o novo índice para cada tentativa, até encontrar um slot com status `EMPTY` ou `DELETED` disponível para inserção. O uso direto do primeiro slot com status `DELETED` encontrado só é possível devido à verificação prévia se a tabela já possui a chave utilizando o método `contains()`. A inserção possui complexidade $O(1)$ no melhor caso e $O(n)$ no pior caso, quando a tabela precisa ser toda percorrida para encontrar um slot disponível.

2.8.3. Remoção

A remoção é realizada pelo método público `remove()`, que utiliza o `aux_search()` para buscar o índice da chave que deve ser removida, e lança uma exceção caso ela não exista. Ao invés de limpar o slot, a remoção marca o slot do índice encontrado como

DELETED, técnica que é conhecida como **lazy deletion**. Essa abordagem é necessária para não interromper a sondagem do double hashing, uma vez que um slot EMPTY interromperia a procura de outros slots cheios além dele. A remoção possui complexidade $O(1)$ no melhor caso e $O(n)$ no pior caso.

2.8.4. Busca

A busca de um valor na tabela de endereçamento aberto também se baseia na sondagem dos slots com base nas tentativas, usando o método privado `aux_search()`. O método percorre a tabela nos slots determinados pelo `hash_code()`, pulando slots com status DELETED, até encontrar a chave e retornar seu índice. Caso encontre um slot com status EMPTY antes de achar a chave, retorna -1 , indicando que a chave não existe na tabela. Essa operação possui complexidade $O(1)$ no seu melhor caso e complexidade $O(n)$ no pior caso.

2.8.5. Iterador

O iterador da tabela com endereçamento aberto é implementado pela classe interna `HashIterator`, que herda da interface `Iterator`. Diferente do iterador da tabela com encadeamento, ele não usa uma fila, mas sim um índice `m_pos` que avança pelo vetor pulando slots com status diferente de ACTIVE, por meio do método privado `advance()`. Os métodos `hasNext()` e `next()` utilizam esse índice para percorrer apenas os slots ativos da tabela, o que diminui consideravelmente o número de slots a ser percorrido.

3. Listagem de Testes Executados e Resultados Encontrados

Nesta seção, são apresentados os testes realizados para validar e comparar o desempenho das estruturas de dados implementadas (Árvore AVL, Árvore Rubro-Negra, Tabela Hash com Encadeamento Exterior e Tabela Hash com Endereçamento Aberto). O sistema foi submetido ao processamento de seis obras literárias de diferentes tamanhos e idiomas, permitindo uma análise abrangente do comportamento de cada algoritmo sob diferentes volumes de dados.

3.1. Listagem de Testes Executados

Os testes consistiram na execução do programa compilado `freq` variando a estrutura escolhida e o arquivo de texto de entrada. A lista completa de arquivos de texto utilizados nos testes é apresentada a seguir:

- **Teste 1:** `o-manifesto-comunista-english.txt` (Volume reduzido)
- **Teste 2:** `dom-casmurro.txt` (Volume médio, língua portuguesa)
- **Teste 3:** `the-secret-garden.txt` (Volume médio, língua inglesa)
- **Teste 4:** `sherlock-holmes.txt` (Volume alto, língua inglesa)
- **Teste 5:** `a-riqueza-das-nacoes-english.txt` (Volume muito alto)
- **Teste 6:** `kjv-bible.txt` (Volume massivo)

O padrão de execução adotado no terminal para mapear o comportamento de cada teste seguiu o modelo abaixo:

```
./freq dicionario_avl caminho/do/livro.txt
./freq dicionario_redblack caminho/do/livro.txt
./freq dicionario_chaining caminho/do/livro.txt
./freq dicionario_openAdr caminho/do/livro.txt
```

Após o término das execuções de cada livro, o script complementar em Python foi acionado para ler os registros gerados em formato tabular e gerar as representações visuais correspondentes:

```
python3 generate_charts.py result/nome\_do\_livro\_Metricas.csv
```

3.2. Resultados e Métricas de Avaliação

Os resultados coletados durante as baterias de testes foram consolidados e mapeados através das métricas descritas a seguir.

3.2.1. Métrica 1: Número de Comparações de Chaves

Esta métrica contabiliza a quantidade total de operações de comparação de strings feitas durante as etapas de busca, inserção ou incremento de frequência das chaves no dicionário. Os gráficos gerados a partir da pasta `graficos` revelam a discrepância assintótica entre as abordagens de árvore e de tabelas hash.

3.3. Análise de Desempenho de cada ED

A análise combinada dos gráficos de comparações e tempo permite traçar o perfil exato de desempenho de cada estrutura de dados:

- **Árvore AVL:** Demonstrou o desempenho mais custoso em volumes massivos de dados (como na Bíblia KJV). Devido à sua propriedade de balanceamento rígido (onde a diferença de altura das subárvores nunca excede 1), cada palavra repetida inserida exige buscas completas do topo à folha que culminam em verificações estruturais e rotações frequentes. Isso gerou as maiores marcas tanto em contagem de comparações quanto em segundos totais de execução.
- **Árvore Rubro-Negra:** Apresentou comportamento assintótico similar ao da AVL ($O(\log n)$), porém com tempos de montagem ligeiramente inferiores em arquivos grandes. Isso ocorre porque o critério de balanceamento da Rubro-Negra é mais flexível (permitindo subárvores com até o dobro do tamanho uma da outra), o que reduz drasticamente o número de rotações e reestruturações de ponteiros durante o fluxo contínuo de modificação do dicionário.
- **Hash com Encadeamento Exterior (HashChaining):** Consistiu em uma das soluções mais eficientes para o cenário de contagem de frequência de palavras. O acesso de complexidade constante ($O(1)$ no caso médio) evitou o acúmulo de comparações de chaves. A alocação dinâmica das listas encadeadas nos buckets mostrou-se resiliente para lidar com o crescimento de termos distintos sem estourar o tempo de processamento.
- **Hash com Endereçamento Aberto (HashOpenAdr):** Obteve tempos excelentes e o menor índice de comparações diretas de chaves nos casos em que o fator de carga se manteve balanceado. Ao utilizar o método de *double hashing* para resolução de conflitos, a estrutura eliminou a necessidade de ponteiros extras e

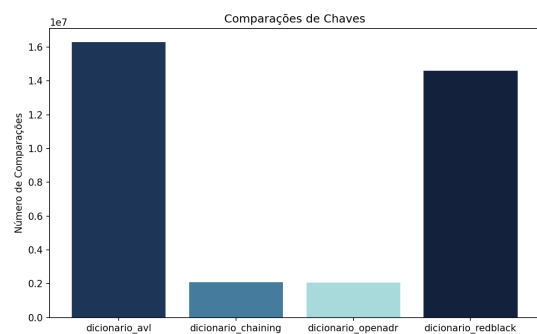
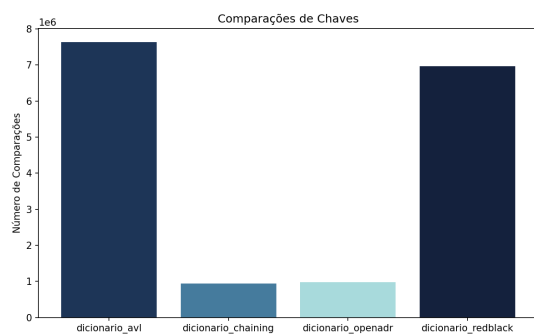
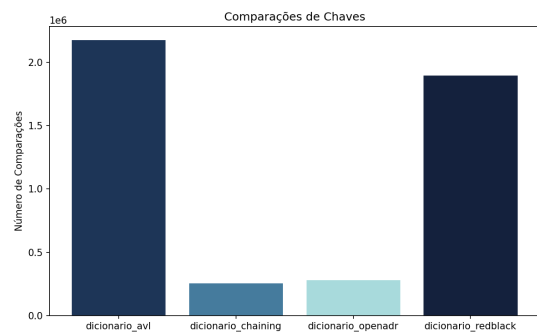
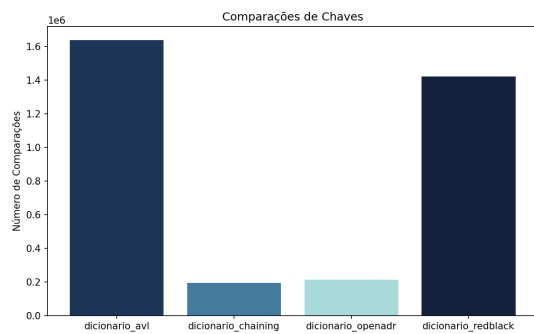
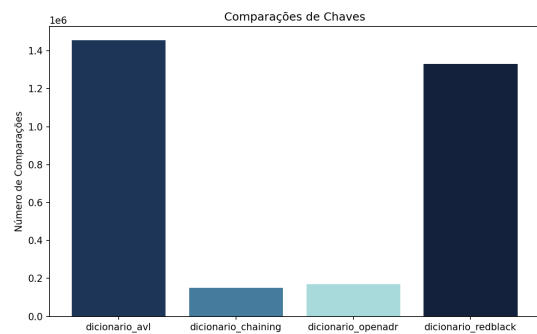
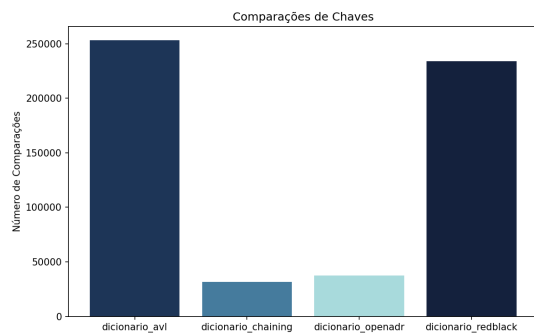


Figura 13. Comparativo do número de comparações de chaves por obra avaliada.

listas encadeadas, otimizando a localidade de referência na memória cache do sistema operacional. Contudo, em livros extensos como *A Riqueza das Nações*, seu desempenho mostrou-se altamente sensível às colisões consecutivas quando a tabela se aproximava do limite do fator de carga máximo estabelecido.

4. Conclusão

Ao decorrer do trabalho, implementamos e comparamos quatro estruturas de dados (Árvore AVL, Árvore Rubro-Negra, Tabela Hash com Encadeamento e Tabela Hash com Endereçamento Aberto) para resolver o problema de contar a frequência de palavras em arquivos de texto. Também foram utilizadas interfaces para padronizar as operações do dicionário e dos iteradores, facilitando a troca da estrutura usada sem alterar o fluxo principal do programa.

Os testes práticos mostraram que as árvores balanceadas não são a melhor escolha para esse tipo de problema. Como os textos têm muitas palavras repetidas, o programa precisa atualizar a contagem o tempo todo. Nas árvores, cada atualização exige descer vários níveis e, muitas vezes, fazer rotações demoradas para manter o balanceamento. A Árvore Rubro-Negra se saiu um pouco melhor que a AVL por ser menos rigorosa e fazer menos rotações, mas ambas foram muito lentas nos livros grandes.

Por outro lado, as Tabelas Hash tiveram um resultado excelente e muito superior. Elas conseguem encontrar e atualizar as palavras quase instantaneamente, fazendo pouquíssimas comparações de chaves. A Hash com Encadeamento lidou muito bem com os livros maiores, mantendo a rapidez mesmo com muitas palavras diferentes. Já a Hash com Endereçamento Aberto também foi muito rápida, mas deixou claro que precisa de espaço sobrando (um bom fator de carga) para não perder desempenho lidando com colisões.

Por fim, concluímos que para criar um dicionário de frequências, as Tabelas Hash são estruturas mais eficazes em relação ao tempo de montagem e o número de comparações de chaves. Elas fazem muito menos comparações e terminam o processamento em um tempo muito menor que as árvores.

5. Divisão das tarefas

Tabela 2. Divisão das tarefas de codificação

Componente	Responsável pela codificação
Interfaces <code>Dictionary</code> e <code>Iterator</code>	Francisco Tiago
Árvore AVL	Francisco Tiago e Wellington
Árvore Rubro-Negra	Francisco Tiago
Tabela Hash com Encadeamento Exterior	Francisco Tiago
Tabela Hash com Endereçamento Aberto	Wellington
Função de hashing FNV-1a	Francisco Tiago e Wellington
Processador de texto <code>Text_Processor</code>	Francisco Tiago e Wellington
Função <code>main</code> e integração	Francisco Tiago e Wellington
Testes, métricas e revisão final	Francisco Tiago e Wellington

Tabela 3. Divisão das tarefas de escrita do relatório

Seção do Relatório	Responsável pela escrita
Listagem de Testes Executados e Resultados Encontrados	Wellington
Conclusão	Francisco Tiago e Wellington
Demais seções (Introdução, Metodologia, Resumo, etc.)	Francisco Tiago

Referências

- Beilis, A. and Grund, A. (2025). Boost.locale - boost c++ libraries. <https://www.boost.org/doc/libs/latest/libs/locale/doc/html/index.html>. Acesso em: 28 de junho de 2026.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. The MIT Press, Cambridge, Massachusetts, 4 edition.
- Szwarcfiter, J. L. and Markenzon, L. (2010). *Estruturas de Dados e Seus Algoritmos*. LTC, Rio de Janeiro, 3 edition.